PORTFOLIO REPORT

LLM Evaluation and Data Quality

A portfolio case study in response evaluation, function-calling quality, and enterprise task design.

CENTRAL RESEARCH QUESTION

How can we evaluate language-model responses and tool-calling workflows consistently, ensuring that models follow user instructions, produce accurate outputs, and complete tasks reliably?

Overview

This project examines how to evaluate language-model outputs and design tasks that can be graded consistently. It combines independent response scoring, tool-dependency analysis, and evidence-based review of enterprise workflows.

The central approach is to extract atomic requirements, ground each judgment in observable evidence, and distinguish critical failures from smaller quality issues. The resulting assessments connect model behavior to explicit criteria and actionable recommendations.

This case study presents completed analysis, not aggregate benchmark results or new model runs. All model scores and decisions are retained from the source report.

How to read the case studies

Each case now follows **task question → requirements → observed behavior → evaluation → takeaway**. Reader-facing question boxes are paraphrases of the existing use-case descriptions. The original complete prompts, full responses, and requirement-level score sheets were not included in the source PDF.

Project scope

Area	Work completed
A/B response evaluation	Three response pairs; weighted requirements, critical-failure gates, and annotation-consistency checks.
Function calling	Two supplied recipe workflows; two added designs testing parallel operations and conditional fallback.
Enterprise task design	Incident response, calendar booking, and headcount reporting reviews; reference task designs and customer-contract guidance.

A/B response evaluation

Evaluation method

Each response is assessed independently before a relative preference is assigned.

- Extract requirements from the system and user prompts.
- Separate requirements into atomic, observable checks.
- Fix the weights and identify the small number of critical gates.
- Score each response against the same requirements.
- Calculate the weighted score and apply any rating cap.
- Lock absolute ratings, then compare the responses.
- Check that evidence, comments, scores, and final preference agree.

The independent-before-comparison approach is informed by PRePair. Weights, thresholds, and gates are project-specific calibration choices rather than an industry standard.

Requirement weights

Weight	Classification	Meaning
5	Critical	Necessary to achieve the main task; failure materially damages correctness or usefulness.
3	Important	Contributes meaningfully, although an imperfect result can remain useful.
1	Minor	A secondary quality or presentation requirement with limited impact on the core task.

Fulfilment scores

Score	Meaning
4	Fully satisfied
3	Satisfied with a minor defect
2	Partially satisfied, with a meaningful issue
1	Major failure, but some relevant attempt
0	Missing, incorrect, or contradicted

WEIGHTED SCORING FORMULA

$$\text{Weighted score (\%)} = 100 \times \Sigma(\text{weight} \times \text{fulfilment score}) \div [4 \times \Sigma(\text{weights})]$$

Absolute ratings and failure gates

Weighted score	Preliminary rating
90-100	Amazing
75-89	Pretty good
55-74	Okay
30-54	Pretty bad
0-29	Horrible

A gate qualifies only when it is necessary for the core goal, failure makes the answer materially wrong or unreliable, other strengths cannot compensate, and the failure is objectively identifiable.

- No gate failure: retain the score-derived rating.
- Serious gate failure (score 0 or 1): the final rating cannot exceed **Okay**.
- Fundamental core-task failure (wrong task, entity, or time period): final rating cannot exceed **Pretty bad**.

These are maximum ratings, not automatic assignments; a lower score-derived rating remains lower.

Relative preference

Condition	Preference
Two or more rating bands apart, **or** one fundamental core-task failure that the other avoids	Much better
Exactly one rating band apart, **or** same band with a weighted-score gap greater than 5 percentage points	Better
Same rating band with a gap of 5 percentage points or less	Tie

Worked scoring example - calculation only

The table below **demonstrates the formula using invented example fulfilment values.** It is not a reconstruction of any of the three reviewed models; the source PDF does not provide their item-level scores.

Example requirement	Weight	Score (0-4)	Points
Task correctness	5	3	15
Evidence quality	3	2	6
Presentation instruction	1	4	4
Total	**9**		**25**

ILLUSTRATIVE RESULT

100 × 25 ÷ (4 × 9) = **69.4% → Okay**. With no gate failure assumed, the score-derived band is retained.

Auditability boundary: The original 45%, 75%, 81%, 88%, 82%, and 68% scores remain as reported. No missing rubric rows, numerical contributions, or verbatim model outputs have been invented to explain them.

Results across the three A/B samples

Sample	Model A	Model B	Final preference
Paris weather prediction	45% Pretty bad	75% Pretty good	Model B is much better
20 pros / 20 cons of owning a cat	81% Pretty good	88% Pretty good	Model B is better
Environmental policy newsletter	82% Pretty good	68% Okay	Model A is better

Case 01 / Paris weather prediction

TASK QUESTION / READER-FACING PARAPHRASE

Will it rain in Paris, France this weekend?

Reconstructed from the report's use-case description; not claimed as the verbatim original prompt.

Requirements. Interpret "this weekend" using the sample's fixed current date; retrieve forecast evidence for Paris, France and the correct period; avoid overstating certainty; begin with the matching weather emoji repeated exactly three times.

Model A	Model B
45% · Pretty bad Used evidence primarily for **Paris, Texas**. This is a fundamental entity-grounding failure.	**75% · Pretty good** Addressed the intended location and supported its conclusion with relevant forecast evidence, but missed a formatting rule.

DECISION AND TAKEAWAY

Model B is much better. Correct entity grounding is indispensable. Strong presentation cannot compensate for forecasting the wrong Paris.

Case 02 / Cat ownership comparison

TASK QUESTION / READER-FACING PARAPHRASE

Provide 20 meaningfully distinct pros and 20 cons of owning a cat, organized in an easy-to-read table.

Reconstructed from the report's use-case description; not claimed as the verbatim original prompt.

Requirements. Include both sets in the requested quantities, keep points relevant and nonredundant, and present a usable comparison.

Observed behavior. Model A supplied 19 pros and 19 cons rather than 20 each, with some thematic repetition. Model B better met the quantity requirement and supplied more informative entries.

Decision. Model B is better (81% vs 88%; both rated Pretty good). The visible count requirement matters even when both responses are broadly useful.

Case 03 / Environmental policy newsletter

TASK QUESTION / READER-FACING PARAPHRASE

Draft an approximately 500-word English-language newsletter for an environmental nonprofit about the EU Carbon Border Adjustment Mechanism (CBAM), its 2026 definitive phase, and implications for French businesses.

Reconstructed from the report's use-case description; not claimed as the verbatim original prompt.

Requirements. Policy accuracy, France-specific relevance, requested length, appropriate sourcing, and professional newsletter tone. The original request was in English and did not request a change of language.

Observed behavior. Model A produced a more directly usable English draft with clearer source transparency. Model B had deeper France-specific discussion but switched to French, ran over length, omitted direct source links in the final article, and left editorial residue. Both responses had factual problems.

DECISION AND TAKEAWAY

Model A is better (82% Pretty good vs 68% Okay). Extra detail does not outweigh language, accuracy, and deliverable requirements.

CBAM findings refer to the reviewed sample; this is not current EU policy guidance.

Annotation quality findings

The annotation review found several consistency problems:

- Explicit instruction failures were sometimes missed or underweighted.
- Stylistic preferences, such as engagement cues or a closing summary, were treated as requirements without prompt support.
- Factual errors and language mismatches received less attention than surface presentation.
- Absolute ratings did not always match defect severity.
- One sample contained contradictory relative-preference selections.

REVIEWER CHECK

For every material criticism, identify **(1) the requirement, (2) the observed evidence, (3) the rating effect**; then verify that narrative, numerical score, and final preference agree.

Reading the findings responsibly

The sample questions above are readable reconstructions, not the complete underlying prompts. The report retains the source's model comparisons and absolute ratings without implying access to raw response transcripts, detailed score sheets, or contemporaneous forecasts.

Function-calling evaluation

The framework separates **structural validity** (can the workflow execute under the actual tool contracts?) from **workflow quality** (does the sequence solve the user request efficiently and consistently?).

Two-layer checklist

Layer	Check	Evaluation question
Structural validity	Tool availability	Is every called function enabled?
Structural validity	Tool contract	Does the call match what the implementation actually does?
Structural validity	Input availability	Do the arguments come from the prompt or earlier outputs?
Workflow quality	Tool fit	Do the tools naturally support the request?
Workflow quality	Workflow logic	Are dependencies and conditional branches meaningful?
Workflow quality	Task completeness	Can the whole user request be satisfied?
Workflow quality	Tool efficiency	Are unnecessary calls avoided?
Workflow quality	QC auditability	Is expected behavior clear enough to grade objectively?

Scoring. Structural checks pass/fail. Each of the five quality dimensions receives 0 (materially unmet), 1 (partially met), or 2 (fully met): 9-10 with no zero dimension = strong; 6-8 without major zero patterns = usable with revision; 0-5 or multiple zeros = weak. Structural failure prevents acceptance as written.

Supplied workflow results

Sample	Structure	Quality	Decision
Recipes within time/budget; four servings	Fail	3/10	Requires redesign
Pancakes for six; flour substitution and groceries	Pass	7/10	Requires targeted revision

Workflow 01 / Time, budget, and serving constraints

TASK QUESTION / READER-FACING PARAPHRASE

Which recipes can feed four people in 30 minutes or less on a $15 budget, with an empty pantry and ingredients estimated at about $3 each?

Reconstructed from the report's use-case description; not claimed as the verbatim original prompt.

Available tools. Only **check_pantry(ingredients, pantry_items)** and **estimate_prep_time(steps)** were enabled.

Observed workflow. The proposed plan required a disabled serving-scaling function. It also lacked an explicit source for recipe steps or ingredients, and the tools could not reliably resolve quantities and cost. Checking an empty pantry offered little informational value.

DECISION · STRUCTURAL FAIL · 3/10

Redesign required. Enable recipe retrieval or supply recipe data; define candidates; remove unsupported scaling assumptions; use a nontrivial pantry; and give budget calculations an explicit basis.

Workflow 02 / Pancakes and gluten-free substitution

TASK QUESTION / READER-FACING PARAPHRASE

Make pancakes for six people using the available butter and sugar; find an appropriate flour substitute for a guest who cannot eat gluten; then return a grocery list.

Reconstructed from the report's use-case description; not claimed as the verbatim original prompt.

Available tools. Recipe retrieval, pantry checking, serving-count scaling, time estimation, and substitution lookup. The evaluation uses the actual supplied implementations.

Observed workflow. Calls used enabled tools and grounded inputs, but **scale_recipe()** logged the new serving count without changing ingredient quantities, and **check_pantry()** tested names rather than amounts. Quantity-aware shopping was therefore unsupported.

The workflow also asked for substitutions outside the requested flour substitution, and needlessly sequenced pantry checking behind scaling, even though that implementation did not modify the ingredient list.

DECISION · STRUCTURAL PASS · 7/10

Targeted revision required. Clarify unsupported quantities; request only the flour substitute; produce one shopping list; and preserve dependencies that actually exist in the tool contracts.

Additional workflow designs

Parallel comparison with conditional substitutions

TASK / AUTHOR-DESIGNED REFERENCE

Determine whether pancakes, stir-fry, or both can be prepared within 50 minutes using listed pantry ingredients and valid substitutes already in that pantry.

- Retrieve both recipes independently.
- For each recipe, estimate prep time and check pantry contents.
- Exclude recipes exceeding 50 minutes.
- For remaining recipes, request substitutes only for missing ingredients.
- Verify that proposed substitutes already exist in the pantry.
- Return all qualifying recipes and substitutions, or report that neither works.

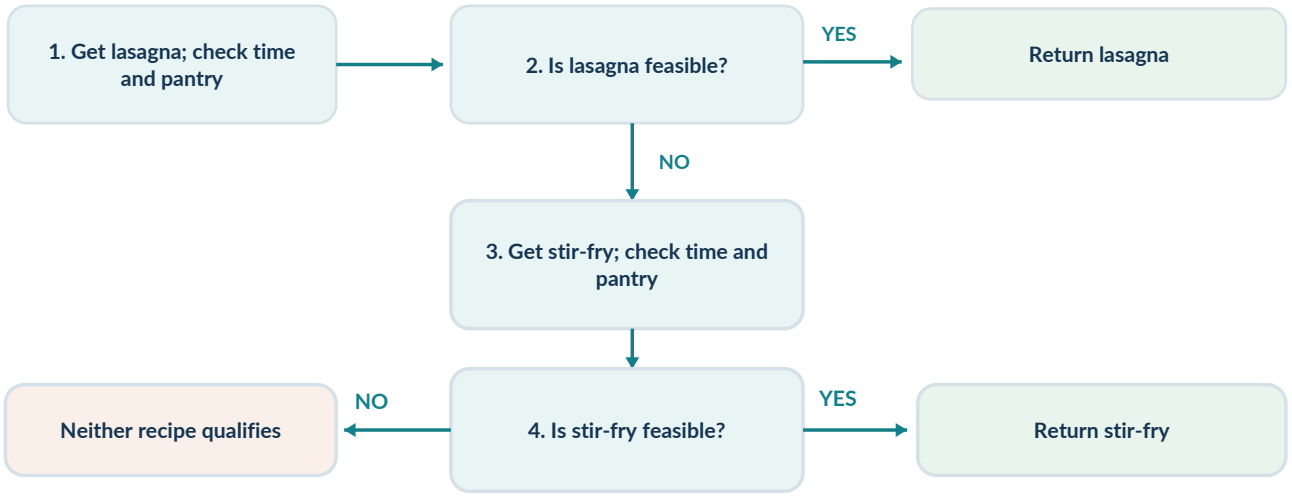
Design assessment: all structural checks passed; 2/2 on every quality dimension. This tests independent calls, shared prerequisites, conditional tool use, and result-dependent validation.

Preferred recipe with conditional fallback

TASK / AUTHOR-DESIGNED REFERENCE

Try lasagna first. Evaluate stir-fry only if lasagna exceeds 50 minutes or cannot be made using pantry ingredients and permitted substitutions.

Illustrative decision path based on the reference workflow design



Logic: Check time and pantry for the preferred recipe. For missing ingredients, only accept valid substitutes already on hand. Return the first feasible recipe; invoke the fallback recipe only after the preferred one fails. If both fail, report neither qualifies.

Design assessment: all structural checks passed; 2/2 on every quality dimension. The diagram summarizes the source's illustrative pseudocode, not a running integration.

Reference pseudocode: preferred recipe fallback

```
def choose_recipe(pantry_items):
    for recipe_name in ["lasagna", "stir-fry"]:
        recipe = get_recipe(recipe_name)
        prep_time = estimate_prep_time(recipe["steps"])

        if prep_time > 50:
            continue

        missing = check_pantry(recipe["ingredients"], pantry_items)
        substitutions = {}
        feasible = True

        for ingredient in missing:
            substitute = suggest_substitute(ingredient)
            if substitute == "No substitute found" or substitute not in pantry_items:
                feasible = False
                break
            substitutions[ingredient] = substitute

        if feasible:
            return {
                "recipe": recipe["name"],
                "estimated_prep_time": prep_time,
                "substitutions": substitutions,
            }

    return "Neither recipe meets the stated constraints."
```

This is the report's illustrative workflow pseudocode using the sample tool contracts, not a standalone executable application.

Enterprise prompt and grading review

Enterprise tasks should combine realistic work with deterministic expected outcomes. The review checks the user request, system rules, available evidence, tools, and final environment state.

Workflow	Main gap	Decision
HTTP 503 incident response	Calendar-time interpretation, current-user identity, and starting incident state underdefined	Revise
Calendar booking	Duration, relative-date anchor, and no-availability fallback underdefined	Revise
Headcount growth dashboard	Year-over-year growth confused with cumulative growth	Revise

Enterprise case 01 / HTTP 503 incident response

TASK QUESTION / READER-FACING PARAPHRASE

A customer-facing application is returning HTTP 503 errors. Coordinate the response using the runbook and connected work tools: alert the team, create and assign tracking work, book a review, draft a customer update without sending it, and log the incident.

Reconstructed from the report's use-case description; not claimed as the verbatim original prompt.

Environment and dependencies. The workflow spans Confluence, Slack, Linear, Calendar, Gmail, and Notion. Backend ownership must be retrieved from the runbook before assignment. Dates, the current user, and existing incident records must be resolved from the environment rather than guessed.

Grading evidence to inspect

- Correct runbook and backend contact were identified.
- Incident notification reached the required channel.
- Tracking project has the required title, label, and retrieved owner.
- Review meeting uses the correct date, timezone, attendee, and description.
- Customer update remains an unsent email draft with required content.
- Incident fields respect defaults and the existing state.
- Returned links and completion claims correspond to real outcomes.

REVIEW FINDING

Revise. A sample incident ID is illustrative, not the next guaranteed ID. Derive the next ID from the current sequence; fix the calendar-time interpretation and current-user identity; make grading criteria atomic.

Task-to-evidence trace

Required action	Evidence of completion
Find incident owner	Retrieved runbook record supports assigned owner
Create / notify / schedule	Resulting project, message, and meeting match instructions
Draft - do not send	Draft exists; no send action occurred
Document incident	Record values reflect current environment and incident sequence

Enterprise case 02 / Calendar booking

TASK QUESTION / READER-FACING PARAPHRASE

Book a meeting for next Friday with three specified colleagues, after identifying all attendees and finding a slot available on all four calendars.

Reconstructed from the report's use-case description; not claimed as the verbatim original prompt.

Requirements. Resolve "next Friday" against a fixed environment date; identify the named colleagues and current user; inspect all four calendars; use a genuinely common valid slot; create the event with correct organizer and attendees.

Defects. Meeting duration, relative-date anchor, and fallback when no common slot exists are missing or unclear. "No common slot" is not the same as every individual being busy.

REVIEW FINDING

Revise. Define duration, working-hours and timezone constraints if applicable, plus what to do if no shared slot exists. Grade correct non-booking when no valid option exists.

Enterprise case 03 / Headcount dashboard

TASK QUESTION / READER-FACING PARAPHRASE

Analyze monthly headcount.csv for Engineering, Marketing, Sales, HR, and Operations from 2020 to 2024. Produce department trends, a stacked area chart, a growth table, and analysis in a Notion report; share the report to #executive-reports.

Reconstructed from the report's use-case description; not claimed as the verbatim original prompt.

Requested outputs. A line chart with department trends, a stacked area chart whose combined layers represent total headcount, a properly defined growth table, narrative analysis, and a published/shared report.

Core defect. The supplied analysis confused **year-over-year growth** with **cumulative growth**. Before recomputing expected numbers, the metric and aggregation basis must be fixed.

Atomic checks for the revised task

- Departments, 2020-2024 period, and month order are correct.
- Each chart has usable labels, legends, and units; separate area layers sum to total headcount.
- Growth calculation follows a precisely stated metric and reference periods.
- Notion page contains both charts, table, and analysis.
- Report link is shared in the requested channel.
- Report title matches the period or the exception is explicitly explained.

REVIEW FINDING

Revise. Translate "executive-ready" into observable output checks and use recalculated ground truth based on a fixed definition.

Reference task designs

These tasks are **author-designed examples** intended to test evidence grounding and deterministic grading; the following briefs are explanatory summaries, not recovered original prompt text.

Reference task 01 / Lead prioritization and outreach drafts

> TASK BRIEF / AUTHOR-DESIGNED
>
> Use a lead spreadsheet and recent email evidence to choose three eligible accounts, create personalized outreach drafts, summarize why the leads were selected, and share the summary. Do not send the drafts.

Eligibility rules: at least 500 employees; not an active customer; lead notes show an explicit business need. Priority signals include a request for next steps, buying intent, unresolved objection, and discussion of the identified need.

Decision rules: a defined evidence window; ranking and recency checks; a final alphabetical tie-break. Each draft must reference supported account evidence, include a next step, and remain unsent.

Grade: eligible selection, evidence basis, ranking, contact accuracy, personalized draft, non-sending, summary completeness, and correct sharing.

Reference task 02 / Unresolved incident summary

> TASK BRIEF / AUTHOR-DESIGNED
>
> Find P0 and P1 incidents within an inclusive reporting window that remain unresolved, then produce a structured, correctly ordered incident summary.

Inclusion: statuses Investigating, In Progress, or Monitoring. **Exclusion:** Resolved and Closed. Capture incident ID, description, priority, status, owner, and creation date.

Sort: oldest to newest by creation date, then ascending incident ID when dates match. Grading checks both inclusion of qualifying incidents and exclusion of non-qualifying incidents.

Why these briefs work as grading tasks

Both define a target population, evidence sources, decision order, output schema, and a concrete definition of successful action and non-action. They reduce the chance that two annotators interpret the same task differently.

Customer-contract annotation guideline

Purpose and scope

Create realistic contract-workflow tasks supported by the environment and consistently gradable. Appropriate cases include renewals, expiry and notice deadlines, owner identification, prioritization, conditional follow-up, and structured reporting.

Exclude unsupported legal conclusions, undefined risk labels, unnecessary tools, or tasks without the evidence needed to complete them.

Sample creation process

Focus	Design requirement
Target population	Specify exactly which contracts or accounts qualify.
Evidence and rules	Anchor decisions in observable facts: dates, statuses, notice periods, support issues, and explicit customer concerns.
Priority and tie-breaks	Define deterministic ranking when more than one candidate qualifies.
Outputs and conditional actions	State exactly what to create and when not to act.
Environment	Verify evidence, ownership, supported tools, and fixed temporal context.
Atomic grading	Check one material requirement per criterion.

Worked grading trace (illustrative structure)

The following is a **rubric-writing illustration**, not an assertion about actual contract records or scored samples.

Instruction	Observable evidence	Pass / fail check
Select contracts with expiry within a fixed reporting window	Contract expiry field + fixed reference date	Each selected record falls in scope
Contact the responsible owner only when a follow-up trigger holds	Ownership record + documented trigger + resulting draft/action	Correct owner; action only when condition is met
Do not flag missing information as risk without a defined rule	Risk rule and available data	No unsupported risk classification

Acceptance criteria

A sample is ready when its **target population and ownership evidence are determinable**; decision and priority rules include unambiguous tie-breaks; conditional actions specify triggers and correct non-action; and its rubric covers material outcomes without requiring unnecessary navigation. Time-dependent instructions must use a fixed date, time, and timezone where needed. Given the same environment, two annotators should reach the same required decisions and final state.

Final decision definitions

Decision	Meaning
Accept	Workflow, evidence, business rules, environment, and grading criteria are complete and deterministic.
Revise	Valid use case, but rules, environment requirements, conditional actions, or grading criteria need clarification.
Reject	Unrealistic or unsupported workflow, or one that cannot be graded reliably.

Key takeaways

- A polished answer can still fail its core task.
- Tool availability and input provenance must be checked before judging workflow quality.
- More tool calls, longer responses, or more sources do not independently establish quality.
- A valid tool call does not prove the entire user request can be completed.
- Reliable grading requires prompt, environment, business rules, and criteria to align.
- Correct non-action is part of successful completion.

Transparency and evidence boundaries

Questions reconstructed from use-case descriptions are explicitly identified as paraphrases. Numerical results are reported from the evaluated samples, while the weighted-score walkthrough and contract grading trace are illustrative only. Detailed score sheets and full model responses are not included in this case study.

References

References retained from the original report for the evaluation approach and tool-use concepts:

Jeong, H., Park, C., Hong, J., Lee, H., & Choo, J. (2025). *The Comparative Trap: Pairwise Comparisons Amplifies Biased Preferences of LLM Evaluators.* Proceedings of the 8th BlackboxNLP Workshop, 79-108. Paper.

Tool use with Claude. Documentation.

Function calling. API documentation.

> **Methodology note:** Reported results refer to the reviewed samples rather than a benchmark. Task questions labeled as paraphrases are not verbatim prompts; worked scoring and grading examples are explanatory, not reconstructed evaluation records.